

Data Structures Using C

Lecture 2: Memory Organization, Pointers and Pointer Arithmetic

Dr. Siddhant Thapliyal

August 1, 2026

Computer Memory Organization

Computer memory can be viewed as a sequence of storage locations.

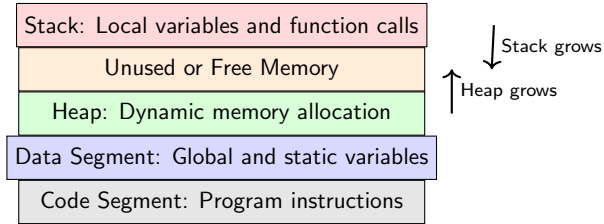
- Every memory location has a unique address.
- Variables occupy one or more consecutive memory locations.
- The allocated memory depends on the data type.
- A variable name provides a convenient way to access stored data.

Important Idea

Every variable has:

- 1 A value
- 2 A memory address

Simplified Memory Layout of a C Program



Note

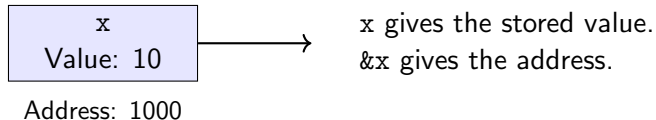
The exact layout depends on the operating system, compiler and hardware architecture.

Variables in Memory

Consider:

```
1 int x = 10;
```

Assume an integer occupies 4 bytes and the starting address of `x` is 1000.



Lvalue and Rvalue

Consider:

$$x = 10;$$

Lvalue

An lvalue refers to a memory location where a value can be stored.

$$x \longrightarrow \text{memory location of } x$$

Rvalue

An rvalue refers to the actual value used in an expression.

$$10 \longrightarrow \text{value assigned to } x$$

Lvalue and Rvalue: Example

Statement	Status	Reason
<code>x = 10;</code>	Valid	<code>x</code> represents a modifiable memory location.
<code>y = x;</code>	Valid	The value stored in <code>x</code> is assigned to <code>y</code> .
<code>10 = x;</code>	Invalid	A constant is not a modifiable memory location.

Simple Rule

The left side of an assignment must represent a modifiable memory location.

What is a Pointer?

Definition

A pointer is a variable that stores the memory address of another variable.

Suppose:

```
int x = 10;
```

and the address of *x* is 1000.

If:

```
int *ptr = &x;
```

then:

ptr = 1000 and **ptr* = 10

Remember

ptr stores an address, while **ptr* accesses the value stored at that address.

Declaring Pointer Variables

General syntax:

```
1 data_type *pointer_name;
```

Examples:

```
1 int      *intPtr;  
2 char     *charPtr;  
3 float    *floatPtr;  
4 double   *doublePtr;
```

- `intPtr` points to an integer.
- `charPtr` points to a character.
- `floatPtr` points to a floating-point value.

Initializing a Pointer

```
1 int x = 10;  
2 int *ptr = &x;
```

Here:

- `x` stores the value 10.
- `&x` gives the address of `x`.
- `ptr` stores the address of `x`.
- `*ptr` accesses the value stored in `x`.

$ptr = \&x$ and $*ptr = x = 10$

Address and Dereference Operators

Operator	Name	Purpose
<code>&</code>	Address operator	Returns the memory address of a variable.
<code>*</code>	Dereference operator	Accesses or modifies the value stored at the address contained in a pointer.

For:

```
int x = 10,      int *ptr = &x
```

we have:

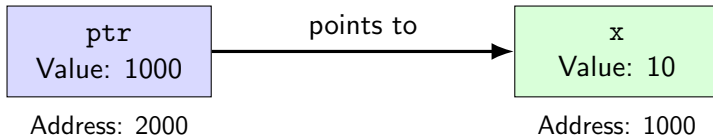
```
ptr = &x      and      *ptr = x
```

Pointer Memory Diagram

Assume:

$x = 10$, $\&x = 1000$

and the address of `ptr` is 2000.



$ptr = 1000$, $*ptr = 10$

Accessing and Modifying Data Through a Pointer

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int x = 10;
6     int *ptr = &x;
7
8     printf("x = %d\n", x);
9     printf("*ptr = %d\n", *ptr);
10
11     *ptr = 25;
12     printf("Updated x = %d\n", x);
13
14     return 0;
15 }
```

Output

```
x = 10
*ptr = 10
Updated x = 25
```



What is Pointer Arithmetic?

C allows a limited set of arithmetic operations on pointers.

- Increment: $ptr++$
- Decrement: $ptr--$
- Add an integer: $ptr + n$
- Subtract an integer: $ptr - n$
- Subtract compatible pointers: $ptr_2 - ptr_1$

Important

Pointer arithmetic is meaningful mainly for elements of the same array.

Pointer Arithmetic Depends on Data Type

Suppose:

*int * ptr*

and an integer occupies 4 bytes.

If:

ptr = 1000

then:

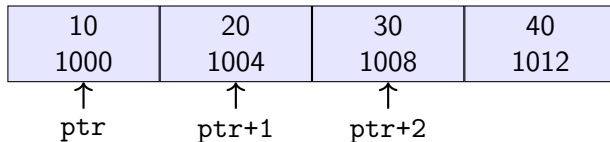
$ptr + 1 = 1004$, $ptr + 2 = 1008$, $ptr + 3 = 1012$

Important

Adding 1 to a pointer moves it to the next element of its type, not necessarily to the next byte.

Pointer Increment Visualization

Assume an integer occupies 4 bytes.



$$*(ptr + 2) = 30$$

Pointer Arithmetic Example

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int numbers[] = {10, 20, 30, 40};
6     int *ptr = numbers;
7
8     printf("%d\n", *ptr);
9     printf("%d\n", *(ptr + 1));
10    printf("%d\n", *(ptr + 2));
11
12    ptr++;
13    printf("%d\n", *ptr);
14
15    return 0;
16 }
```

Output

10, 20, 30, 20

Valid and Invalid Pointer Operations

Expression	Status	Meaning
<code>ptr++</code>	Valid	Move to the next element.
<code>ptr--</code>	Valid	Move to the previous element.
<code>ptr + 3</code>	Valid	Move three elements forward.
<code>ptr2 - ptr1</code>	Conditional	Distance between elements of the same array.
<code>ptr1 + ptr2</code>	Invalid	Two addresses cannot be added.
<code>ptr * 2</code>	Invalid	Pointer multiplication is not allowed.

Operator Precedence with Pointers

Operator precedence determines which operation is performed first.

Consider:

$$*ptr ++$$

The postfix increment operator `++` has higher precedence than the dereference operator `*`.

Therefore:

$$*ptr ++ \equiv *(ptr ++)$$

Meaning

The current value is accessed first, and then the pointer moves to the next element.

Pointer Expressions and Precedence

Suppose:

```
int x = 10,    int * ptr = &x
```

Expression	Meaning
<code>*ptr</code>	Access the value pointed to by ptr.
<code>ptr++</code>	Move the pointer to the next element.
<code>(*ptr)++</code>	Increment the value pointed to by ptr.
<code>*ptr++</code>	Access the current value, then increment the pointer.
<code>++*ptr</code>	Increment the pointed value first, then use it.

Understanding Pointer Expressions

Assume:

```
int arr[] = {10, 20, 30};    int * ptr = arr;
```

- `*ptr` gives 10.
- `(*ptr)++` changes the first element from 10 to 11.
- `ptr++` makes `ptr` point to 20.
- `*ptr++` accesses 10 and then moves the pointer to 20.

Best Practice

Use parentheses whenever a pointer expression may be misunderstood.

Relationship Between Arrays and Pointers

For:

$$\text{int arr}[5] = \{10, 20, 30, 40, 50\}$$

the array name represents the address of its first element.

$$\text{arr} = \&\text{arr}[0]$$

Therefore:

$$\text{arr}[0] = *\text{arr}$$
$$\text{arr}[1] = *(\text{arr} + 1)$$
$$\text{arr}[i] = *(\text{arr} + i)$$

Traversing an Array Using a Pointer

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int arr[] = {10, 20, 30, 40, 50};
6     int *ptr = arr;
7     int i;
8
9     for (i = 0; i < 5; i++)
10         printf("%d ", *(ptr + i));
11
12     return 0;
13 }
```

Output

10 20 30 40 50

NULL Pointer

A NULL pointer does not point to any valid memory location.

```
1 int *ptr = NULL;
```

Before dereferencing, check whether it is valid.

```
1 if (ptr != NULL)
2 {
3     printf("%d", *ptr);
4 }
```

Warning

Dereferencing a NULL pointer may cause a runtime error or program crash.

Void Pointer or Generic Pointer

A void pointer can store the address of any data type.

```
1 int x = 10;  
2 void *ptr = &x;  
3  
4 printf("%d", *(int *)ptr);
```

- A void pointer has no specific data type.
- It must be typecast before dereferencing.
- It is useful in generic functions.
- Dynamic memory allocation functions return void pointers.

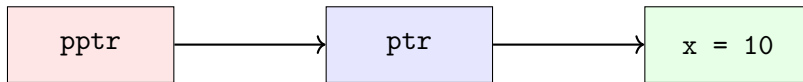
Pointer to Pointer

A pointer can store the address of another pointer.

```
1 int x = 10;  
2 int *ptr = &x;  
3 int **pptr = &ptr;
```

Then:

$x = 10,$ $*ptr = 10,$ $**pptr = 10$



Common Pointer Errors

Uninitialized pointer

```
1 int *ptr;  
2 *ptr = 20;
```

Correct initialization

```
1 int x;  
2 int *ptr = &x;
```

Assigning a value

```
1 int x = 10;  
2 int *ptr;  
3 ptr = x;    // Wrong
```

Correct assignment

```
1 ptr = &x;
```

Common Pointer Problems

Problem	Meaning
Dangling pointer	Points to memory that has already been released.
Wild pointer	An uninitialized pointer containing an unknown address.
NULL pointer	A pointer intentionally initialized to point nowhere.
Memory leak	Allocated memory is not released after use.
Out-of-bounds access	The pointer moves outside a valid array boundary.

Safety Rule

Dereference a pointer only when it contains a valid and accessible address.

Applications of Pointers

- Passing arguments by reference
- Modifying variables inside functions
- Passing arrays and strings
- Dynamic memory allocation
- Efficient array traversal
- Creating linked lists
- Implementing stacks and queues
- Constructing trees and graphs
- Returning multiple results
- Building dynamic data structures

Example: Passing by Reference

```
1 #include <stdio.h>
2
3 void swap(int *a, int *b)
4 {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
10 int main(void)
11 {
12     int x = 10, y = 20;
13     swap(&x, &y);
14     printf("x = %d, y = %d", x, y);
15     return 0;
16 }
```

Output

x = 20, y = 10

Quick Classroom Quiz

- 1 What does a pointer store?
- 2 What is the difference between `&x` and `*ptr`?
- 3 What is the difference between an lvalue and an rvalue?
- 4 If an integer pointer stores 1000, what is its value after `ptr++`, assuming an integer occupies 4 bytes?
- 5 What is the difference between `ptr++` and `(*ptr)++`?

Predict the Output

Given:

```
int arr[] = {5, 10, 15, 20};    int *ptr = arr;
```

Predict:

```
printf( "%d" ,*(ptr + 2));
```

Predict the Output

Given:

```
int arr[] = {5, 10, 15, 20};    int *ptr = arr;
```

Predict:

```
printf( "%d" , *(ptr + 2));
```

Answer

15

because $*(ptr + 2) = arr[2]$.

Practice Questions

- 1 Write a program to add two numbers using pointers.
- 2 Find the largest array element using pointer arithmetic.
- 3 Reverse an array using pointers.
- 4 Differentiate between a pointer and an ordinary variable.
- 5 Explain `*ptr`, `&x`, `ptr+1`, and `(*ptr)++`.

Questions?

Thank You